Assignment 3: Sentiment Analysis using Feedforward Neural Networks

Jackie Javier, Pranitha Achanta, Robert McDaniels
The University of New Mexico
CS429

Due 4/10/2026

1 Introduction

Sentiment analysis is a Natural Language Processing (NLP) task that involves determining whether a piece of text expresses a positive or negative opinion. In this project, we performed sentiment analysis on the IMDb movie review dataset using Feedforward Neural Networks (FNNs). The textual data was first transformed into numerical representations using Term Frequency-Inverse document Frequency (TF-IDF) vectors with the `sklearn.feature_extraction.text` module. These vectors were then used as input features for training neural network models. We explored several machine learning techniques to improve model performance, including baseline FNN modeling, hyperparameter tuning, k-fold cross validation, dropout regularization, and ensemble learning. The goal of this project was to achieve high classification accuracy ($\geq$90%) while analyzing the trade-offs between model complexity and computational cost. Reports of "time costs" are given, with the terminology being interpreted as the time taken for completing all steps of the algorithm, not just pure training times. Pure training times are printed separately within the code for reference.

2 Data Preparation

The IMDb movie review dataset was used for sentiment classification. Each review was labeled as either positive (1) or negative (0).

2.1 Text Preprocessing

Before transforming the text into numerical features, several preprocessing steps were applied to clean the data:

- Conversion of all text to lowercase
- Removal of punctuation and special characters

- Removal of common stopwords (e.g., "the", "and", "is")

These steps help reduce noise and improve the quality of the extracted features.

2.2 Feature Extraction

The cleaned text data was transformed into numerical feature vectors using the Term Frequency–Inverse Document Frequency (TF-IDF) method implements in `TfidfVectorizer` from `sklearn.feature_extraction.text`. TF-IDF assigns weights to words based on their importance in a document relative to the entire dataset. Frequently occurring words in a specific review receive higher weights, while common words across all documents receive lower weights. To control the dimensionality of the feature space, a maximum number of features was specified (20,000 most important terms).

Since TF-IDF produces sparse matrices, the data was converted into dense format and then into PyTorch tensors for model training.

2.3 Train-Test Split

The dataset was divided into:

- 70% training data
- 30% test data

3 Tasks 2 and 3 - FNN Model and Hyperparameter Tuning

3.1 Model Architecture

A Feedforward Neural Network (FNN) was implemented using PyTorch. The model takes TF-IDF vectors as input and outputs a binary sentiment prediction. The architecture of the model is as follows:

- Input layer: TF-IDF feature vector
- Hidden Layer 1: 256 neurons, ReLU activation
- Hidden Layer 2: 64 neurons, ReLU activation
- Hidden Layer 3: 8 neurons, ReLU activation
- Output Layer: 1 neuron (logit output)

The output layer does not use a sigmoid activation explicitly, as the loss function `BCEWithLogitsLoss` internally applies the sigmoid function for numerical stability.

3.2 Training Configuration

The model was trained using the following hyperparameters:

- Hidden Layer Neurons: [256, 64, 8]
- Learning rate: 0.00005
- Weight decay (L2 regularization): 0.0002
- Loss function: BCEWithLogitsLoss
- Batch size: 128
- Number of epochs: 10

The Adam optimizer was chosen due to its faster convergence and better performance compared to standard stochastic gradient descent. The learning rate controls the step size taken during optimization. A lower learning rate was chosen to ensure stable convergence. During tuning, trying a larger learning rate increased the training accuracy, but simultaneously decreased the test accuracy, contributing to overfitting. Weight decay helps prevent overfitting by penalizing large weights in the model. This encourages simpler models that generalize better to unseen data. A somewhat small value was selected to provide mild regularization without significantly restricting the model's capacity. For the number of neurons per hidden layer, the first layer being a larger layer allows the model to capture richer feature interactions from the tfidf input, while the smaller subsequent layers gradually compress the learned representation. This architecture balances expressive power with generalization, mirroring the constrained approach taken towards tuning the learning rate and weight decay.

3.3 Baseline Results and Comparison

The performance of the FNN model was compared with a logistic regression model from the textbook.

Model	Training Accuracy	Test Accuracy	Time Cost
Logistic Regression	0.897	0.899	3.22 sec
FNN (Baseline)	0.9714	0.9068	69.48 sec

Table 1: Comparison between Logistic Regression and FNN

The FNN model achieved a test accuracy of 90.68%, successfully meeting the project requirement of achieving at least 90% accuracy. However, this improved performance comes at a significantly higher computational cost. The FNN requires substantially more training time compared to logistic regression, due to the increased model complexity and iterative training over multiple epochs.

4 Task 4 - k-Fold Cross Validation

Model	Training Accuracy	Test Accuracy	Time Cost
Baseline FNN	0.9714	0.9068	69.48 sec
k-Fold FNN	0.9778	0.9093	373.71 sec

Table 2: Performance Comparison: Baseline vs k-Fold Cross Validation

The k-fold cross validation approach provides a more reliable estimate of model performance by reducing the impact of data splitting variability. The average validation accuracy (0.9093) is very close to the baseline test accuracy (0.9068), indicating that the model generalizes well. Note that the metric used for the "Test Accuracy" comparison **is indeed** the average validation accuracy, rather than the average voting on the test set. As k-fold creates training and validation partitions from the entire dataset (train + test), if the models are trained using the full dataset, the train and test accuracies become almost equivalent:

```
Avg Train Accuracy: 0.9743, Train Time: 411.34 sec
Avg Validation Accuracy: 0.9058, Validation Time: 1.50 sec
Test Accuracy (Avg Predictions): 0.9703, Test Time: 2.73 sec
```

If we instead limit the training and validation data to only the training set, the 30% of the data in the test set remains unseen. As such, each model performs slightly worse than the baseline model, as they have less data to train from:

```
Avg Train Accuracy: 0.9615, Train Time: 240.90 sec
Avg Validation Accuracy: 0.8795, Validation Time: 0.44 sec
Test Accuracy (Avg Predictions): 0.8896, Test Time: 2.72 sec
```

These attempts do not constitute an evaluation of the baseline model's reliability. This understanding is congruent with the class slides, and the given tutorial in the assignment.

In k-fold cross validation, the training data is divided into $k = 5$ equals subsets. For each iteration, one subset is used as validation data while the remaining subsets are used for training. This process is repeated five times so that each subset is used once for validation. The final performance is obtained by averaging the results across all folds.

However, k-fold cross validation significantly increases the computational cost, with total training time increasing from 69.48 to 373.71 seconds. This demonstrates the trade-off between computational efficiency and robustness of model evaluation.

5 Task 5 - Dropout and Ensemble

Dropout stochastically zeroes activations with a given constant probability to encourage model sparsity and regularization. Neurons are forced to learn independent, redundant pathways to compensate for being unable to rely on inputs.

Properly tuned, it can improve the test accuracy and generalization. In task 5.1, we demonstrate a straightforward application of this principle.

Boost aggregating (bagging) trains "base models" on subsets of the total training data with replacement, then combines them into an ensemble model. In task 5.2, we combine dropout with bagging (mean-vote using subsampling *with replacement*) and demonstrate that it can marginally improve the final test accuracy.
NOTE: The instructions mention "bagging" with no other specifics given. As such, we chose to use the most generic form: generating 5 new sets by sampling from the test set with replacement. This means that the sets given to each model are the same size as the original training set (subsets are not required to be smaller than the full set). As such, training time is not improved.

Each base model is exposed to less of the data overall and thus has less data to overfit on. However, because subsampling results in some samples being overrepresented, these tend to overfit in different directions. The aggregate effect of ensemble voting cancels these errors and results in lower overall variance than a single model.

5.1 Ensemble Architecture

In order to make the most of the ensemble, we varied the sizes of the base models and the dropout probability for all layers:

- Hidden Layer Neurons: [256, 64, 8], Dropout 0.5
- Hidden Layer Neurons: [256, 128, 64], Dropout 0.6
- Hidden Layer Neurons: [128, 64, 16], Dropout 0.5
- Hidden Layer Neurons: [512, 128, 16], Dropout: 0.6
- Hidden Layer Neurons: [256, 32, 8], Dropout: 0.5

5.2 Results and Comparison

Model	Training Accuracy	Test Accuracy	Time Cost
Baseline	0.9714	0.9068	69.48 sec
Dropout	0.9694	0.9106	68.00 sec
Ensemble	0.9628	0.9117	343.48 sec

Table 3: Comparison between dropout models

Compared to the baseline, dropout decreases the training accuracy (which we generally don't care about) but modestly increases the final testing accuracy as well as training slightly faster.

For the ensemble results, we tried many different configurations; a few did outperform the single dropout, but not consistently or by a statistically significant margin. This is generally because the models aren't different enough from each other to bring in novel insights, and accuracy is fundamentally limited by the information lost in the tf-idf transformation (order and context).

All models share the same learning rate/ weight decay (instructed to do so by the professor during office hours), number of hidden layers, tf-idf features, and non-disjoint training data. As a result, they tend to learn similar decision boundaries and repeat the same mistakes. Additionally, the baseline model already achieves a high test accuracy, meaning it is nearing the performance ceiling; ensemble methods are typically more effective when combining weaker or moderately accurate models, so any improvements here are small and unstable.

More models with more diverse structures would improve ensemble performance.